CONTENTS

5.b		Write a Java program that accepts a string from the user and counts the number of vowels, consonants, digits, and special characters present in it.		
6		Design and Develop an Simple Calculator using AWT Event Handling.		
7		Design a Student Marks File Management System that stores, retrieves, and updates student marks using Java File Handling		
8		Create a Student Registration Form with fields like name, roll number, and course using Java Swing components (Java GUI Practice – Student Registration Form).		
9		Design a Student Database Management System that performs CRUD operations on student records using Java and JDBC (Java Practice – Student Database Management System).		
10		Design and develop a Student Record Management System using ArrayList and HashMap to perform Add, Update, Search, Delete, and Display operations on student records		
BEYOND THE SYLLABUS				
A		Design and implement a Chess Game using Object-Oriented Programming principles such as inheritance, polymorphism, and encapsulation to model pieces, moves, and game rules.		
B		Design and implement an Employee Data Analytics program that uses Java Stream API and Lambda Expressions to filter, sort, group, and summarize employee data		

C		Design and implement a Generic Data Container using a Generic Class, Generic Method, and Bounded Type Parameters to store and process data of multiple types safely		

Date :	**Constructors and Static Members**
Exp No: 1.a	
LeetCode ID 1603 Question : Design Parking System Design a parking system for a parking lot with a fixed number of spaces for big, medium, and small cars.	

Aim:

To design and implement a class ParkingSystem that manages parking of three kinds of vehicles — **big**, **medium**, and **small** — using object-oriented programming concepts, and to check whether a vehicle can be parked based on the number of available slots

Algorithm:

1. Define a class ParkingSystem with three instance variables bigSlots, mediumSlots, and smallSlots to track available parking spaces of each type.
2. In the constructor, initialize these three variables with the values passed by the user, so every object created gets its own independent slot count.
3. Define a private helper concept: map carType (1 = big, 2 = medium, 3 = small) directly to the corresponding slot variable — no separate hash function is needed here since car types are fixed categories.
4. For addCar(carType): check the slot count corresponding to that carType; if it is greater than 0, decrement it by 1 and return true, else return false.
5. For getAvailableSlots(carType) (helper for testing/menu): return the current count of slots left for that type without modifying it.
6. For resetSystem(big, medium, small) (optional helper): reinitialize the slot counts, useful for testing multiple scenarios in one run.
7. Maintain a static counter objectCount that increments every time a new ParkingSystem object is created, to track total objects created.
8. Test the class by creating an object, calling addCar for different vehicle types in sequence, and verifying the output.

Procedure:

1. Open a Java IDE or text editor and create a file named ParkingSystemDemo.java; type/paste the program code.
2. Save the file and compile it using javac ParkingSystemDemo.java.
3. Run the program using java ParkingSystemDemo.
4. Observe the constructor message confirming object creation and initial slot counts for big, medium, and small vehicles.
5. Use the menu to perform operations: add a car (specifying type), view available slots for a type, reset the system with new slot counts, or view total objects created (static member).

6. Enter the required input (car type: 1, 2, or 3) as prompted for each addCar operation.
7. Repeat for multiple car types to verify that slots decrease correctly and addCar returns false once a type's slots are exhausted.
8. Choose the exit option to terminate the program.
9. Record the output and verify it matches the expected ParkingSystem behavior.

Description:

Design a parking system for a parking lot. The parking lot has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.
Implement the ParkingSystem class:

ParkingSystem(int big, int medium, int small) Initializes object of the ParkingSystem class.The number of slots for each parking space are given as part of the constructor.
bool addCar(int carType) Checks whether there is a parking space of carType for the car that wants to get into the parking lot. carType can be of three kinds: big, medium, or small, which are represented by 1, 2, and 3 respectively. A car can only park in a parking space of its carType. If there is no space available, return false, else park the car in that size space and return true .

Example I/P & O/P:

Input:

["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]

[[1, 1, 0], [1], [2], [3], [1]]

Output

[null, true, true, false, false]

Explanation

ParkingSystem parkingSystem = new ParkingSystem(1, 1, 0);

parkingSystem.addCar(1); // return true because there is 1 available slot for a big car

parkingSystem.addCar(2); // return true because there is 1 available slot for a medium car

parkingSystem.addCar(3); // return false because there is no available slot for a small car

parkingSystem.addCar(1); // return false because there is no available slot for a big car. It is already occupied.

Constraints:

- 0 <= big, medium, small <= 1000
- carType is 1, 2, or 3
- At most 1000 calls will be made to addCar

Result:

Date :	**Constructors and Static Members**
Exp No: 1.b	
LeetCode ID 705 Question : Design HashSet Design a HashSet without using any built-in hash table libraries.	

Aim:

To design and implement a class My HashSet in Java that supports add(key), remove(key), and contains(key) operations without using Java's built-in hash-based collections, and to understand the role of constructors in initializing instance state and static members in class design.

Algorithm:

1. Define a class My HashSet with a fixed bucket array size (a prime number, e.g., 10007) to minimize collisions.
2. In the constructor, initialize an array of LinkedList<Integer>, one linked list per bucket, so every object created gets its own independent storage.
3. Define a private helper method hash(key) that maps a given key to a bucket index using the modulo operator (key % SIZE).
4. For add(key): compute the bucket index using hash(key); if the key is not already present in that bucket's list, insert it.
5. For remove(key): compute the bucket index using hash(key); remove the key from that bucket's list if it exists (cast to Integer to remove by value, not index).
6. For contains(key): compute the bucket index using hash(key); check whether the key exists in that bucket's list and return true/false.
7. Test the class by creating an object, calling add, remove, and contains in sequence, and verifying the output.

Procedure:

1. Open a Java IDE or text editor and create a file named HashSetDemo.java; type/paste the program code.
2. Save the file and compile it using javac HashSetDemo.java.
3. Run the program using java HashSetDemo.
4. Observe the constructor message confirming object creation and bucket initialization.
5. Use the menu to perform operations: add a key, remove a key, check if a key exists, display all keys, or view total objects created (static member).
6. Enter the required input (key values) as prompted for each operation.

7. Repeat for multiple keys to verify add, remove, and contains operations work correctly.
8. Choose the exit option to terminate the program.
9. Record the output and verify it matches the expected HashSet behavior.

Description:

Design a HashSet without using any built-in hash table libraries.

Implement MyHashSet class:

- void add(key) Inserts the value key into the HashSet.
- bool contains(key) Returns whether the value key exists in the HashSet or not.
- void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.

Sample I/P & O/P:

Example 1:
Input
["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove", "contains"]
[[], [1], [2], [1], [3], [2], [2], [2], [2]]
Output
[null, null, null, true, false, null, true, null, false]

Explanation
MyHashSet myHashSet = new MyHashSet();
myHashSet.add(1); // set = [1]
myHashSet.add(2); // set = [1, 2]
myHashSet.contains(1); // return True
myHashSet.contains(3); // return False, (not found)
myHashSet.add(2); // set = [1, 2]
myHashSet.contains(2); // return True
myHashSet.remove(2); // set = [1]
myHashSet.contains(2); // return False, (already removed)

Constraints:

- $0 <= key <= 10^6$
- At most 10^4 calls will be made to add, remove, and contains.

Result:

Date :	**Inheritance**
Exp No: 2	
LeetCode ID 1396 Question : Design Underground System Underground Railway System to track customer travel times between stations	

Aim:

To design and implement a class Underground System in Java that supports check In(id, station Name, t), check Out(id, station Name, t), and get Average Time (start Station, end Station) operations, and to understand the role of constructors in initializing instance state and static members in class design.

Algorithm:

1. Define a class UndergroundSystem with two HashMaps: one to store ongoing check-ins (mapping customer id to station name and check-in time), and another to store travel statistics (mapping a route key to total time and total trip count).
2. In the constructor, initialize both HashMaps so every object starts with empty check-in records and empty route statistics.
3. For checkIn(id, stationName, t): store the customer's id along with the station name and check-in time in the check-in map.
4. For checkOut(id, stationName, t): retrieve the customer's check-in record using id, compute the travel time as (checkout time − checkin time), form a route key from startStation and endStation, and update the total time and trip count for that route.
5. For getAverageTime(startStation, endStation): form the route key, fetch the total time and trip count for that route, and return total time divided by trip count.
6. Maintain a static counter to track how many UndergroundSystem objects have been created, and a static method to retrieve this count without needing an object.
7. Test the class using a menu-driven program that takes station names, ids, and times as input from the user and displays results accordingly.

Procedure:

1. Open a Java IDE or text editor and create a file named HashSetDemo.java; type/paste the program code.
2. Save the file and compile it using javac HashSetDemo.java.
3. Run the program using java HashSetDemo.
4. Observe the constructor message confirming object creation and bucket initialization.
5. Use the menu to perform operations: add a key, remove a key, check if a key exists, display all keys, or view total objects created (static member).
6. Enter the required input (key values) as prompted for each operation.
7. Repeat for multiple keys to verify add, remove, and contains operations work correctly.

8. Choose the exit option to terminate the program.
9. Record the output and verify it matches the expected HashSet behavior.

Description:

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the UndergroundSystem class:

- void checkIn(int id, string stationName, int t)
  A customer with a card ID equal to id, checks in at the station stationName at time t.
  A customer can only be checked into one place at a time.
- void checkOut(int id, string stationName, int t)
  A customer with a card ID equal to id, checks out from the station stationName at time t.
- double getAverageTime(string startStation, string endStation)
  1. Returns the average time it takes to travel from startStation to endStation.
  2. The average time is computed from all the previous traveling times from startStation to endStation that happened directly, meaning a check in at startStation followed by a check out from endStation.
  3. The time it takes to travel from startStation to endStation may be different from the time it takes to travel from endStation to startStation.
  4. There will be at least one customer that has traveled from startStation to endStation before getAverageTime is called.

You may assume all calls to the checkIn and checkOut methods are consistent. If a customer checks in at time t_1 then checks out at time t_2, then $t_1 < t_2$. All events happen in chronological order.

Sample I/P & O/P:

Input

["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut","getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]

[[],[45,"Leyton",3],[32,"Paradise",8],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Cambridge",22],["Paradise","Cambridge"],["Leyton","Waterloo"],[10,"Leyton",24],["Leyton","Waterloo"],[10,"Waterloo",38],["Leyton","Waterloo"]]

Output

[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]

Explanation

UndergroundSystem undergroundSystem = new UndergroundSystem();

undergroundSystem.checkIn(45, "Leyton", 3);

undergroundSystem.checkIn(32, "Paradise", 8);

undergroundSystem.checkIn(27, "Leyton", 10);

undergroundSystem.checkOut(45, "Waterloo", 15); // Customer 45 "Leyton" -> "Waterloo" in 15-3 = 12

undergroundSystem.checkOut(27, "Waterloo", 20); // Customer 27 "Leyton" -> "Waterloo" in 20-10

undergroundSystem.checkOut(32, "Cambridge", 22); // Customer 32 "Paradise" -> "Cambridge" in 22-8 = 14

undergroundSystem.getAverageTime("Paradise", "Cambridge"); // return 14.00000. One trip "Paradise" -> "Cambridge", (14) / 1 = 14

undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 11.00000. Two trips "Leyton" -> "Waterloo", (10 + 12) / 2 = 11

undergroundSystem.checkIn(10, "Leyton", 24);

undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 11.00000

undergroundSystem.checkOut(10, "Waterloo", 38); // Customer 10 "Leyton" -> "Waterloo" in 38-24 = 14

undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 12.00000. Three trips "Leyton" -> "Waterloo", (10 + 12 + 14) / 3 = 12

.

Constraints:

- $1 <= id, t <= 10^6$
- 1 <= stationName.length, startStation.length, endStation.length <= 10
- All strings consist of uppercase and lowercase English letters and digits.
- There will be at most $2 * 10^4$ calls **in total** to checkIn, checkOut, and getAverageTime.
- Answers within 10^{-5} of the actual value will be accepted.

Result:

Date :	**Packages And Interfaces With Access Specifiers**
Exp No: 3.a	
A Java Shape Area Calculator demonstrating packages, interfaces, and access specifiers.	

Aim:

To design and implement a Shape Area Calculator in Java using the concepts of Packages, Interfaces, and Access Specifiers .Where each shape class provides its own specific implementation of the calculate Area() method defined in the Shape interface.

Algorithm:

1. Create a package named geometry.
2. Create an interface Shape containing the method calculateArea().
3. Create three classes namely Circle, Rectangle, and Triangle inside the package.
4. Implement the Shape interface in all the classes.
5. Use appropriate access specifiers (private, protected, and public) for data members and methods.
6. Create constructors to initialize the dimensions of each shape.
7. Override the calculateArea() method in every class.
8. Create a main class outside the package.
9. Import the package and create objects of different shapes.
10. Display the calculated area of each shape.

Procedure:

1. Open a Java IDE or text editor and create a file named HashSetDemo.java; type/paste the program code.
2. Save the file and compile it using javac HashSetDemo.java.
3. Run the program using java HashSetDemo.
4. Observe the constructor message confirming object creation and bucket initialization.
5. Use the menu to perform operations: add a key, remove a key, check if a key exists, display all keys, or view total objects created (static member).

6. Enter the required input (key values) as prompted for each operation.
7. Repeat for multiple keys to verify add, remove, and contains operations work correctly.
8. Choose the exit option to terminate the program.
9. Record the output and verify it matches the expected HashSet behavior.

Sample Output

----- Shape Area Calculator -----

Circle:
Radius = 5.0
Area of Circle = 78.53981633974483

Rectangle:
Length = 4.0, Width = 6.0
Area of Rectangle = 24.0

Triangle:
Base = 3.0, Height = 8.0
Area of Triangle = 12.0

Result:

Date :	**Packages And Interfaces With Access Specifiers**
Exp No: 3.b	
Design a class that computes and stores the value of Pi using public, private, and protected access specifiers to control member access .	

Aim:

To write a Java program that calculates the value of Pi using a class with member variables and methods declared under different access specifiers (private, public, protected, and default), thereby demonstrating the concept of access control in Object-Oriented Programming.

Algorithm:

1. Create a class PiCalculator.
2. Declare a private variable to store the radius (or number of terms, depending on method used) accessible only within the class.
3. Declare a public final variable/constant PI (or compute it) — accessible from outside the class.
4. Declare a protected method calculateArea(double radius) that uses PI to compute the area of a circle accessible within the same package/subclasses.
5. Declare a public method getPiValue() that returns the value of Pi — accessible from anywhere, including main.
6. In the constructor, initialize the private radius variable.
7. In the main method (in a different/driver class or same class):
   a. Create an object of PiCalculator.
   b. Access the public method to display the value of Pi.
   c. Try accessing the private variable directly (to demonstrate it is **not** accessible — shown as a comment or compile-time error explanation).
   d. Call the protected/public method to calculate and display the area using Pi.

Procedure:

1. Open a Java IDE or text editor and create a file named PiCalculator.java.
2. Type/paste the program code demonstrating private, protected, and public members.
3. Save the file and compile it using javac PiCalculator.java.
4. Run the program using java PiCalculator.
5. Observe that the public members (Pi value, area) are accessible directly from main/driver class.
6. Attempt to access the private variable from outside the class (comment this line in code) and note the compilation error, confirming access control.
7. Enter the radius value as prompted to compute the area using Pi.
8. Record the output.
9. Verify that access specifiers behave as expected (private hidden, public/protected accessible appropriately).

Sample Output

```
=== Pi Calculator using Access Specifiers ===

Enter the number of terms for Pi approximation: 5

Calculating Pi using Leibniz Series...

Public Method - Displaying Result:
Approximated value of Pi: 3.3396825396825403

Protected Method - Displaying Precision Info:
Precision used: 5 terms
Series used: Leibniz Series (4/1 - 4/3 + 4/5 - 4/7 + 4/9 ...)

Private Data - Accessed only within class:
Raw computed value (private): 3.3396825396825403
```

Result:

Date :	**Packages And Interfaces With Access Specifiers**
Exp No: 3.c	
LeetCode ID 1232 Question : Check If It Is a Straight Line Given an array of coordinates, check whether these points make a straight line in the XY plane	

Aim:

To design a program that determines whether a given set of points (given as [x, y] coordinates) all lie on the same **straight line**, using the concept of slope comparison.

Algorithm:

1. Read the array of coordinates, where each element is a pair [x, y].
2. If the number of points is 2 or fewer, return true directly (two points always form a line).
3. Take the first two points as reference: (x0, y0) and (x1, y1).
4. Compute the direction vector: dx = x1 - x0, dy = y1 - y0.
5. For every subsequent point (x, y) in the array (from index 2 onward):
   a. Compute dxCurr = x - x0, dyCurr = y - y0.
   b. Check the cross-product condition: dx * dyCurr - dy * dxCurr == 0.
   (This avoids division and floating-point errors that a slope-based dy/dx comparison would cause.)
   c. If the condition is not satisfied for any point, return false immediately.
6. If all points satisfy the condition, return true.

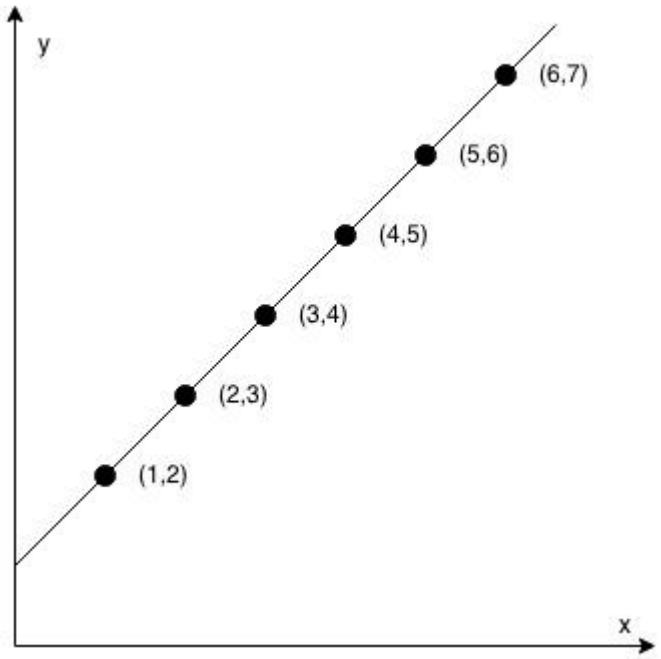
Procedure:

1. Open a Java IDE or text editor and create a file named CheckStraightLine.java.
2. Type/paste the program code implementing the class Solution with method checkStraightLine(int[][] coordinates).
3. Save the file and compile it using javac CheckStraightLine.java.
4. Run the program using java CheckStraightLine.
5. Provide/hardcode a 2D array of coordinate points as input.
6. Observe the constructor/driver output for the boolean result.
7. Test with multiple sets of points — some forming a straight line and some not.
8. Record the output for each test case.
9. Verify the results against expected outcomes.

Sample I/P & O/P:

-You are given an integer array coordinates, coordinates[i] = [x, y], where [x, y] represents the coordinate of a point. Check if these points make a straight line in the XY plane.

Example 1:



Input: coordinates = [[1,2],[2,3],[3,4],[4,5],[5,6],[6,7]]

Output: true

Constraints:

- 2 <= coordinates.length <= 1000
- coordinates[i].length == 2
- -10^4 <= coordinates[i][0], coordinates[i][1] <= 10^4
- coordinates contains no duplicate point.

Result:

Date :	**User-Defined Exception and Multiple Threads**
Exp No: 4	
LeetCode ID 1114 Question : Print in Order Ensure three threads execute in a fixed sequence so their output is always ordered	

Aim:

To design and implement a class Foo in Java whose three methods first(), second(), and third() are invoked by three separate threads, and to guarantee — using synchronization — that the output always appears in the order first → second → third regardless of the order in which the threads are launched, while using a user-defined exception to validate the thread-launch order supplied by the user.

Algorithm:

1. Define a user-defined exception class InvalidThreadOrderException by extending the Exception class, with a constructor that passes a message to the superclass.
2. Define a class Foo containing two CountDownLatch objects, each initialized to 1, to act as gates between the ordered method calls.
3. In first(): print "first", then release the latch that second() is waiting on (countDown()).
4. In second(): wait on the first latch (await()), print "second", then release the latch that third() is waiting on.
5. In third(): wait on the second latch (await()), then print "third".
6. Define a method validateOrder() that checks the user-supplied launch order is exactly the numbers 1, 2, 3 with no repeats and no invalid values; throw InvalidThreadOrderException otherwise.
7. In main(), read the launch order, validate it (catching the custom exception), create three threads bound to first(), second(), third(), and start them in the user-specified order.
8. Join all threads and confirm that the printed output is still in the correct sequence..

Procedure:

1. Open a Java IDE or text editor and create a file named PrintInOrder.java; type/paste the program code.
2. Save the file and compile it using javac PrintInOrder.java.
3. Run the program using java PrintInOrder.
4. When prompted, enter the order in which the three threads should be launched (for example 3 1 2).
5. Observe that even though the threads are started out of order, the synchronization gates force the output to appear as first, second, third.
6. Run the program again and enter an invalid order (a repeated number, a value outside 1–3, or

the wrong count) to trigger the user-defined exception.

7. Verify that Invalid Thread Order Exception is caught and its message is displayed instead of running the threads.
8. Record the output and confirm it matches the expected ordered behaviour for valid input and the exception message for invalid input.

Sample I/P & O/P:

Suppose we have a class:

```
public class Foo {
  public void first() { print("first"); }
  public void second() { print("second"); }
  public void third() { print("third"); }
}
```

The same instance of Foo will be passed to three different threads. Thread A will call first(), thread B will call second(), and thread C will call third(). Design a mechanism and modify the program to ensure that second() is executed after first(), and third() is executed after second().

Note:

We do not know how the threads will be scheduled in the operating system, even though the numbers in the input seem to imply the ordering. The input format you see is mainly to ensure our tests' comprehensiveness.

Example 1:

Input: nums = [1,2,3]

Output: "firstsecondthird"

Explanation: There are three threads being fired asynchronously. The input [1,2,3] means thread A calls first(), thread B calls second(), and thread C calls third(). "firstsecondthird" is the correct output.

Constraints:

- nums is a permutation of [1, 2, 3].

Result:

Date :	**String Handling**
Exp No: 5.a	
LeetCode ID 242 Question : Valid Anagram Determine whether two strings are anagrams of each other, i.e. one is a rearrangement of the other.	

Aim:

To design and implement a Java program that checks whether two given strings are anagrams of each other using string-handling operations, without relying on any built-in anagram utility, and to reinforce the use of String, StringBuilder, character arrays, and frequency counting in string processing..

Algorithm:

1. Read two strings s and t from the user.
2. Normalise both strings by converting them to lower case and removing any spaces so the comparison is case-insensitive and space-insensitive.
3. If the lengths of the two normalised strings differ, they cannot be anagrams — return false immediately.
4. Create an integer frequency array of size 26 (one slot per lowercase letter).
5. Traverse the first string and increment the count for each character; traverse the second string and decrement the count for each character.
6. After processing, if every slot in the frequency array is zero, the strings are anagrams; otherwise they are not.
7. As an alternative verification, convert both strings to character arrays, sort them, rebuild them using StringBuilder, and check whether the two sorted strings are equal.
8. Display the result of both the frequency-count method and the sorting method.

Procedure:

1. Open a Java IDE or text editor and create a file named ValidAnagram.java; type/paste the program code.
2. Save the file and compile it using javac ValidAnagram.java.
3. Run the program using java ValidAnagram.
4. When prompted, enter the first string and then the second string.
5. Observe that the program normalises the input (lowercase, spaces removed) before comparing, so case and spacing do not affect the result.
6. Verify that both the frequency-count method and the sorting method report the same conclusion.
7. Repeat with different inputs — try a valid anagram pair (e.g. listen / silent), a non-anagram pair, and strings of different lengths — to confirm correct behaviour in each case.
8. Record the output and confirm it matches the expected anagram result for every test.

Sample I/P & O/P:

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Constraints:

- 1 <= s.length, t.length <= $5 * 10^4$
- s and t consist of lowercase English letters.

.

Result:

Date :	**String Handling**
Exp No: 5.b	
Write a Java program that accepts a string from the user and counts the number of vowels, consonants, digits, and special characters present in it (Java Practice – Character Type Counter).	

Aim:

To write a Java program that accepts a string from the user and counts the number of **vowels**, **consonants**, **digits**, and **special characters** present in it

Algorithm:

1. Start.
2. Read a string input from the user using Scanner.
3. Initialize four counters: vowels = 0, consonants = 0, digits = 0, specialChars = 0.
4. Convert the string to lowercase (or handle both cases) for easier vowel/consonant checking.
5. Traverse the string character by character:
   a. If the character is a letter (a-z or A-Z):
      a. If it is one of a, e, i, o, u, increment vowels.
      b. Else, increment consonants.
      b. Else if the character is a digit (0-9), increment digits.
      c. Else if the character is not a whitespace, increment specialChars.
6. After traversing the entire string, display the counts of vowels, consonants, digits, and special characters.

Procedure:

1. Open a Java IDE or text editor and create a file named CountCharacters.java.
2. Type/paste the program code.
3. Save the file and compile it using javac CountCharacters.java.
4. Run the program using java CountCharacters.
5. Enter a string (containing letters, digits, and special characters) when prompted.
6. Observe the output showing the count of vowels, consonants, digits, and special characters.
7. Repeat with different strings (e.g., only letters, only digits, mixed) to verify correctness.
8. Record the output.
9. Verify the results against manual counting.

Sample Output:
Enter a string: Hello World 123!

Vowels: 3
Consonants: 7
Digits: 3
Special Characters: 2

Breakdown for "Hello World 123!":

- Vowels: e, o, o → 3
- Consonants: H, l, l, W, r, l, d → 7
- Digits: 1, 2, 3 → 3
- Special Characters: space (x2), ! → 2 (spaces are often excluded depending on how you define "special characters" — some programs count only symbols like !@#$% and exclude whitespace)

Result:

Date :	**EVENT HANDLING**
Exp No: 6	
Design and Develop a Simple Calculator using AWT Event Handling.	

Aim:

To write a Java AWT program to demonstrate event handling using the ActionListener interface by developing a Simple Calculator.

Algorithm:

1. Import the required packages java.awt.* and java.awt.event.*.
2. Create a class that extends Frame and implements the ActionListener interface.
3. Add two text fields to accept input numbers.
4. Add five buttons: **Addition, Subtraction, Multiplication, Division,** and **Clear**.
5. Register all buttons using the addActionListener() method.
6. Override the actionPerformed() method to perform the selected arithmetic operation.
7. Display the result in a label.
8. Handle invalid inputs and division by zero using exception handling.
9. Add a WindowAdapter to close the application.
10. Execute the program and verify the output.

Procedure:

1. Open a Java IDE or text editor, create SimpleCalculator.java, type/paste the program code, and compile it using javac SimpleCalculator.java.
2. Run the program using java SimpleCalculator and observe the AWT window opening with two text fields and five buttons.
3. Enter the first number in the first text field and the second number in the second text field.
4. Click any operation button (Addition, Subtraction, Multiplication, or Division) to trigger the actionPerformed() method and perform the calculation.
5. Observe the result displayed in the label, and click Clear to reset the fields for fresh input.
6. Test invalid inputs and division by zero to verify exception handling displays appropriate error messages.
7. Close the window using the WindowAdapter to terminate the application, and record the output for verification.

Sample Output

```
  Case 1
--------------------------------------
                  Simple Calculator
--------------------------------------

  First Number : 25

Second Number : 15

Click : Add

  Result : 40.0
```

```
  Case 2
--------------------------------------
                  Simple Calculator
--------------------------------------

  First Number

: 18 Second Number

: 6 Click : Divide

  Result : 3.0
```

```
First Number : 25

Second Number : 0

Click : Divide

Result : Cannot divide by zero
```

Result:

Date :	**Files I/O**
Exp No: 7	
Design a Student Marks File Management System that stores, retrieves, and updates student marks using Java File Handling (Java Practice – Student Marks File Management System)	

Aim:

To write a Java program to demonstrate file input and output operations by storing, reading, and searching student records using FileWriter, BufferedWriter, FileReader and BufferedReader.

Algorithm:

1. Create a text file named **students.txt**.
2. Accept the details of three students such as Roll Number, Name, and Marks.
3. Write the student details into the file using BufferedWriter.
4. Close the file after writing.
5. Open the same file using BufferedReader.
6. Read and display all the student records.
7. Search for a student using the Roll Number.
8. Display the student details if found; otherwise display an appropriate message.
9. Close the file after reading.

Procedure:

1. Open a Java IDE or text editor, create a file named StudentFile.java, and type/paste the program code.
2. Import the required package java.io.* for FileWriter, BufferedWriter, FileReader, BufferedReader, and IOException.
3. In the program, create a FileWriter wrapped in a BufferedWriter to write to a text file named students.txt.
4. Accept or define the details of three students — Roll Number, Name, and Marks.
5. Write each student's details into the file using the BufferedWriter, then flush and close the writer.
6. Compile the program using javac StudentFile.java.
7. Run the program using java StudentFile.
8. Open the same file students.txt using a FileReader wrapped in a BufferedReader.
9. Read the records line by line using readLine() in a loop and display all the student records.
10. Search for a particular student by entering a Roll Number and comparing it with the records read from the file.
11. Display the matching student's details if found; otherwise, display an appropriate "not found"

message.

12. Close the BufferedReader after reading, handling any IOException using a try-catch block.
13. Verify the output against the expected sample output and record it for verification.

Sample Output

```
Enter Details of 3 Students

Student 1
Roll No : 101
Name : Rahul
Marks : 87

Student 2
Roll No : 102
Name : Sneha
Marks : 91

Student 3
Roll No : 103
Name : Kiran
Marks : 79

Student records saved successfully.
----- Student Records
-----101,Rahul,87
102,Sneha,91
103,Kiran,79

Enter Roll Number to Search : 102

Student Found

Roll No : 102
Name  : Sneha
Marks : 91
```

Result:

Date :	**Swing Application**
Exp No: 8	
Create a Student Registration Form with fields like name, roll number, and course using Java Swing components (Java GUI Practice – Student Registration Form).	

Aim:

To write a Java Swing program to design a Student Registration Form and demonstrate the use of Swing components, event handling, and input validation.

Algorithm:

1. Import the required Swing and AWT packages.
2. Create a class that extends JFrame and implements the ActionListener interface.
3. Add labels, text fields, radio buttons, combo box, check boxes, buttons, and a text area.
4. Group the radio buttons using ButtonGroup.
5. Register the **Submit** and **Clear** buttons using ActionListener.
6. Read the entered details when the **Submit** button is clicked.
7. Validate that mandatory fields are not empty.
8. Display the entered information in the text area.
9. Clear all fields when the **Clear** button is clicked.
10. Execute the program and verify the output.

Procedure:

1. Open a Java IDE or text editor, create a file named StudentRegistration.java, and type/paste the program code.
2. Import the required packages javax.swing.* and java.awt.* along with java.awt.event.* for the Swing components and event handling.
3. Define a class that extends JFrame and implements the ActionListener interface.
4. Add the Swing components — labels, text fields, radio buttons, a combo box, check boxes, buttons, and a text area — and place them on the frame using a suitable layout.
5. Group the gender radio buttons using a ButtonGroup so that only one can be selected at a time.
6. Register the Submit and Clear buttons with the ActionListener using addActionListener(this).
7. Compile the program using javac StudentRegistration.java.
8. Run the program using java StudentRegistration and observe the Student Registration Form window opening with all the components.
9. Enter the student details — fill in the text fields, select a radio button, choose an item from the combo box, and tick the required check boxes.
10. Click the Submit button to trigger the actionPerformed() method, which reads the entered details.
11. Observe the input validation that checks whether the mandatory fields are empty and displays an appropriate message if so.

12. Observe the entered information displayed in the text area when validation passes.
13. Click the Clear button to reset all fields for fresh input.
14. Close the window to terminate the application, and record the output for verification.

Sample Output

Initial Window:

```
-------------------------------------------------------
          STUDENT REGISTRATION FORM
-------------------------------------------------------

USN         :

Name        :

Branch      : [Computer Science
            ▼]
Gender      : ( ) Male    ( )
            Female
Skills      : [ ] Java    [ ] Python

               [ Submit ]       [
              Clear ]
-------------------------------------------------------
Student Details

-------------------------------------------------------
```

Result:

Date :	**DataBase Connections and Operations**
Exp No: 9	
Design a Student Database Management System that performs CRUD operations on student records using Java and JDBC (Java Practice – Student Database Management System).	

Aim:

To write a Java program to connect to a MySQL database using JDBC and perform **Insert, Update, Search, and Display** operations using PreparedStatement.

Algorithm:

1. Create a MySQL database named **college**.
2. Create a table named **student** with Roll Number, Name, Department, and Marks.
3. Load the MySQL JDBC driver.
4. Establish a connection with the database.
5. Create PreparedStatement objects for Insert, Update, Search, and Display operations.
6. Insert two student records into the database.
7. Update the marks of a student using the Roll Number.
8. Search for a student using the Roll Number.
9. Display all student records.
10. Close the database connection.

Procedure:

1. Start the MySQL server and open the MySQL command line (or Workbench). Create a database named college and a table named student with columns Roll Number, Name, Department, and Marks.
2. Ensure the MySQL JDBC driver (Connector/J .jar) is downloaded and added to the project's classpath.
3. Open a Java IDE or text editor, create a file named StudentDB.java, and type/paste the program code.
4. Import the required package java.sql.* for Connection, PreparedStatement, ResultSet, and DriverManager.
5. Load the MySQL JDBC driver using Class.forName("com.mysql.cj.jdbc.Driver").
6. Establish a connection to the college database using DriverManager.getConnection() with the correct URL, username, and password.
7. Create PreparedStatement objects for the Insert, Update, Search, and Display operations, using placeholders (?) for parameter values.
8. Compile the program using javac StudentDB.java.
9. Run the program using java -cp .;mysql-connector-j.jar StudentDB (use : instead of ; on Linux/Mac).

10. Observe the insertion of two student records into the database.
11. Update the marks of a student using the Roll Number and verify the change.
12. Search for a student by Roll Number and display the retrieved record.
13. Display all student records from the table.
14. Close the PreparedStatement and Connection objects.

Sample Output

Database Table

```
CREATE DATABASE college;

USE college;

CREATE TABLE student
(
    rollno INT PRIMARY KEY,
    name VARCHAR(30),
    department VARCHAR(20),
    marks INT
 );
```

Compilation

```
javac -cp .;mysql-connector-j-8.x.x.jar StudentJDBC.java

java -cp .;mysql-connector-j-8.x.x.jar StudentJDBC
```

Sample Output

```
 Records Inserted
 Successfully.

Record Updated Successfully.

Student Details

Roll No : 101
Name    : Rahul
Department : CSE
Marks   : 95

Student Records
-----------------------------------------
Roll    Name    Department      Marks
-----------------------------------------
101     Rahul  CSE              95
102     Sneha  ISE              91
```

Result:

Date :	**ArrayList and Map Interfaces**
Exp No: 10	

Design and develop a Student Record Management System using ArrayList and HashMap to perform Add, Update, Search, Delete, and Display operations on student records (Java Practice – Student Record Management System using Collections).

Aim:

To write a Java program to demonstrate the use of **ArrayList** and **HashMap** by implementing a Student Record Management System.

Algorithm:

1. Create a Student class with Roll Number, Name, and Percentage as data members.
2. Create an ArrayList to store student objects.
3. Create a HashMap to store student records using Roll Number as the key.
4. Add student records to both the ArrayList and HashMap.
5. Display all student records stored in the ArrayList.
6. Search for a student using the Roll Number from the HashMap.
7. Remove a student record from the ArrayList.
8. Display the updated student records.
9. Display all entries stored in the HashMap.

Procedure:

1. Open a Java IDE or text editor, create a file named StudentRecord.java, and type/paste the program code.
2. Import the required package java.util.* for ArrayList, HashMap, and related collection classes.
3. Define a Student class with the data members Roll Number, Name, and Percentage, a constructor to initialize them, and a toString() method to display the details.
4. In the main class, create an ArrayList to store Student objects and a HashMap with Roll Number as the key and the Student object as the value.
5. Add several student records to both the ArrayList and the HashMap.
6. Compile the program using javac StudentRecord.java.
7. Run the program using java StudentRecord.
8. Observe the display of all student records stored in the ArrayList.
9. Search for a particular student by entering a Roll Number and retrieving the record from the HashMap.
10. Remove a student record from the ArrayList and display the updated list.
11. Display all key-value entries stored in the HashMap to verify the records.
12. Verify the output against the expected sample output and record it for verification.

Sample Output:

```
Student Records (ArrayList)

---------------------------------
Roll    Name    Percentage
---------------------------------
101     Rahul  88.5
```

```
102     Sneha  91.2
103     Kiran  84.8

Searching for Roll No : 102

Record Found

Roll No    : 102
Name       : Sneha
Percentage : 91.2

After Removing First Student

---------------------------------
Roll    Name    Percentage
---------------------------------
102     Sneha  91.2
103     Kiran  84.8

Student Records (HashMap)

---------------------------------
101 -> Rahul (88.5%)
102 -> Sneha (91.2%)
103 -> Kiran (84.8%)
```

Result:

Date :	**Chess Game Using OOP**
Exp No: A	

Design and implement a Chess Game using Object-Oriented Programming principles such as inheritance, polymorphism, and encapsulation to model pieces, moves, and game rules (Java Practice – Chess Game using OOP).

Aim:

To design and implement a Chess Game using Object-Oriented Programming (OOP) concepts in Java by modeling the game using classes, objects, inheritance, abstraction, encapsulation and polymorphism.

Algorithm:

1. Create classes to represent the chess components such as Board, Spot, Piece, Player, Move, and Game.
2. Define an abstract Piece class and extend it to create different chess pieces like King, Queen, Rook, Bishop, Knight, and Pawn.
3. Initialize the chessboard with all pieces in their default positions.
4. Accept the player's move and validate it according to the movement rules of the selected chess piece.
5. Update the board if the move is valid; otherwise, reject the move.
6. Alternate turns between the two players until the game ends.
7. Display the game status and declare the winner when the game is completed.

Procedure:

1. Open the Java IDE (Eclipse/IntelliJ IDEA/NetBeans).
2. Create a new Java project.
3. Create the required classes such as Board, Spot, Piece, Player, Move, and Game.
4. Implement the abstract Piece class and extend it for all chess pieces.
5. Initialize the chessboard with all chess pieces.
6. Implement the game logic for validating and executing moves.
7. Compile the program and resolve any compilation errors.
8. Execute the program.
9. Verify the movement of pieces and the game flow.

Sample Output:

```
=== Chess Game (Console Version) ===

Initial Board Setup:
8  r  n  b  q  k  b  n  r
7  p  p  p  p  p  p  p  p
6  .  .  .  .  .  .  .  .
5  .  .  .  .  .  .  .  .
4  .  .  .  .  .  .  .  .
3  .  .  .  .  .  .  .  .
2  P  P  P  P  P  P  P  P
1  R  N  B  Q  K  B  N  R
   a  b  c  d  e  f  g  h

White's turn.
Enter move (e.g., e2 e4): e2 e4
Pawn moved from e2 to e4.

8  r  n  b  q  k  b  n  r
7  p  p  p  p  p  p  p  p
6  .  .  .  .  .  .  .  .
5  .  .  .  .  .  .  .  .
4  .  .  .  .  P  .  .  .
3  .  .  .  .  .  .  .  .
2  P  P  P  P  .  P  P  P
1  R  N  B  Q  K  B  N  R
   a  b  c  d  e  f  g  h

Black's turn.
Enter move (e.g., e7 e5): e7 e5
Pawn moved from e7 to e5.

...

Invalid move attempt:
Enter move: e4 e5
Error: Pawn cannot move diagonally without capturing.

Check!
White's King is in check by Black's Bishop.

Checkmate!
Black wins the game.
```

Result:

Date :	**Lambda Expressions and Stream API**
Exp No: B	
Design and implement an Employee Data Analytics program that uses Java Stream API and Lambda Expressions to filter, sort, group, and summarize employee data (Java Practice – Employee Data Analytics using Streams and Lambdas).	

Aim:

To write a Java program to demonstrate functional programming using lambda expressions, functional interfaces, method references, and the Stream API by performing analytics on employee data.

Algorithm:

1. Create an Employee class with id, name, department, and salary.
2. Store employee objects in a List.
3. Use filter() and sorted() with lambda expressions to list high-salary employees.
4. Use map() with a method reference to extract employee names.
5. Use Collectors.groupingBy() to group employees by department.
6. Use Collectors.averagingDouble() to find the average salary per department.
7. Use reduce() to compute the total salary.
8. Use count(), max(), and Optional to find the CSE count and the highest-paid employee.
9. Display all results.

Procedure:

1. Open a text editor or IDE and create a new file named EmployeeAnalytics.java.
2. Import the required packages: java.util.* and java.util.stream.*.
3. Define the Employee class with the fields id, name, department, and salary, a constructor to initialize them, and an overridden toString() method.
4. In the EmployeeAnalytics class, inside main(), create a List of Employee objects using Arrays.asList().
5. Use forEach() with a lambda to print all employees.
6. Apply stream().filter().sorted().forEach() with lambda expressions to display employees earning above 50000 in descending order.
7. Use map() and Collectors.toList() to collect employee names.
8. Use Collectors.groupingBy() with Collectors.mapping() to group employee names by department.
9. Use Collectors.groupingBy() with Collectors.averagingDouble() to compute the average salary per department.
10. Use reduce() with the Double::sum method reference to compute the total salary.
11. Use filter() and count() to count CSE employees, and max() with Comparator and Optional to find the highest-paid employee.
12. Save the file, then compile it using javac EmployeeAnalytics.java.

13. Run the program using java EmployeeAnalytics.
14. Verify the output against the expected sample output.

Sample Output:

```
---- All Employees ----
101     Rahul  CSE    55000.0
102     Sneha  ECE    62000.0
103     Kiran  CSE    48000.0
104     Divya  MECH   51000.0
105     Arjun  ECE    70000.0

---- Salary Above 50000 (High to Low) ----
Arjun -> 70000.0
Sneha -> 62000.0
Rahul -> 55000.0
Divya -> 51000.0

---- Employee Names ----
[Rahul, Sneha, Kiran, Divya, Arjun]

---- Employees Grouped by Department ----
CSE : [Rahul, Kiran]
ECE : [Sneha, Arjun]
MECH : [Divya]

---- Average Salary per Department ----
CSE : 51500.00
ECE : 66000.00
MECH : 51000.00

Total Salary Paid : 286000.00
Number of CSE Employees : 2
Highest Paid : Arjun (70000.0)
```

Result:

Date :	**Generics**
Exp No: C	
Design and implement a Generic Data Container using a Generic Class, Generic Method, and Bounded Type Parameters to store and process data of multiple types safely (Java Practice – Generic Data Container).	

Aim:

To write a Java program to demonstrate generics by implementing a generic class, a multi-type generic class, and a generic method with a bounded type parameter.

Algorithm:

1. Create a generic class Box<T> that can store and return an item of any type.
2. Create a generic class Pair<K, V> that stores a key-value pair of two different types.
3. Create a generic method findMax() with a bounded type parameter <T extends Comparable<T>>.
4. In main(), create Box objects for Integer and String.
5. Create Pair objects with different type combinations.
6. Call findMax() on Integer, String, and Double arrays.
7. Display all results.

Procedure:

1. Open a text editor or IDE and create a new file named GenericDemo.java.
2. Define a generic class Box<T> with a private field item, a set() method, a get() method, and a showType() method that prints the runtime type of the stored item.
3. Define a generic class Pair<K, V> with two private fields key and value, a constructor, and a display() method.
4. In the GenericDemo class, define a generic method findMax() with a bounded type parameter <T extends Comparable<T>> that returns the maximum element of an array using compareTo().
5. Inside main(), create a Box<Integer> object, store a value using set(), and display it with get() and showType().
6. Create a Box<String> object and repeat the store-and-display steps.
7. Create two Pair objects with different type combinations (e.g., Pair<String, Integer> and Pair<Integer, String>) and call display() on each.
8. Create Integer, String, and Double arrays and pass each to findMax() to display the maximum value.
9. Save the file, then compile it using javac GenericDemo.java.
10. Run the program using java GenericDemo.
11. Verify the output against the expected sample output.

Sample Output:

```
Integer Box Value : 100
Type of stored item : java.lang.Integer
String Box Value : Hello Generics
Type of stored item : java.lang.String

---- Key-Value Pairs ----
Rahul = 88
101 = CSE

Maximum Number : 89
Maximum (Alphabetical) : Sneha
Maximum Marks : 92.3
```

Program :

Result: